

Simulator UAR – code review

Znalezione błędy

BŁĄD 1

- Wskaźnik zapomina gdzie w pamięci komputera znajduje się obiekt wykresu. Wywołanie clear() na niczym powoduje naruszenie pamięci, więc system od razu zamyka program.

```
442 void MainWindow::zresetujSymulacje()
443 {
444     timerSymulacji->stop();
445     czyDziala = false;
446     btnStartStop->setText("Start");
447
448     aktualnyCzas = 0.0;
449     czasBazy = 0.0;
450
451     arx.zresetuj_stan();
452     pid.zresetuj();
453
454     //seriaZadana = nullptr; 1 błąd |
455
456     seriaZadana->clear();
457     seriaWyjscie->clear();
458     seriaUchyb->clear();
459     seriaSterowanie->clear();
460     seriaP->clear();
461     seriaI->clear();
462     seriaD->clear();
463
464     auto resetujOsX = [&](QValueAxis *ax) {
465         ax->setRange(0, this->oknoCzasowe);
466         ax->setTickCount(11);
467     };
```

Znalezione błędy

BŁĄD 2

- Czas był ustawiony na sztywno na 1s. Na ekranie miało na przykład 200 milisekund, a wewnątrz wzorów mijała cała sekunda. Czas na ekranie nie zgadzał się z czasem w obliczeniach, przez co wykresy pokazywały nieprawdę.

```
309 void MainWindow::krokSymulacji()
310 {
311     double dt_gui = interwalMs / 1000.0;
312
313     //double dt_math = 1.0;
314
315     double czasRzeczywistyTarget = czasBazy + (licznikCzasuRzeczywistego.elapsed() / 1000.0);
316
317     double ost_w = 0.0;
318     double ost_y = 0.0;
319     double ost_e = 0.0;
320
321     int maxKrokow = 20;
322     int wykonaneKroki = 0;
323
324     while (aktualnyCzas < czasRzeczywistyTarget && wykonaneKroki < maxKrokow) {
325         aktualnyCzas += dt_gui;
326
327         ost_w = gen.generuj(aktualnyCzas);
328
329         ost_y = petla.wykonaj_krok(ost_w, dt_gui); // zamiast dt_gui bylo dt_math
330
331         ost_e = ost_w - ost_y;
332
333         wykonaneKroki++;
334     }
335
336     if (aktualnyCzas < czasRzeczywistyTarget - dt_gui) {
337         aktualnyCzas = czasRzeczywistyTarget;
338     }
339
340     if (wykonaneKroki > 0) {
341         double u = pid.pobierzOstatnieP() + pid.pobierzOstatnieI() + pid.pobierzOstatnieD();
342         aktualizujDaneWykresow(aktualnyCzas, ost_w, ost_y, ost_e, u);
343     }
344 }
```

Raport z werfikacji kodu

- Wykresy dla stanu ustalonego wyglądają podobnie, ale różnią się w kolejnych cyklach
- Błędny reset: wyzwolenie resetu tylko dla części całkującej błędnie resetuje cały regulator.
- Próbkki nie są zapominane po określonym czasie
- Nadmiarowość generatora: moduł posiada 3 tryby pracy zamiast wymaganych 2
- Brak ograniczeń: w programie nie zaimplementowano możliwości nakładania ograniczeń
- Wątpliwa architektura 3 warstwowa: w warstwie usług znajduje się 1 metoda którą ponadto powielają 2 klasy
- Brak płynnej zmiany okna
- Regulator PID nie działa stabilnie dla parametrów wzorcowych



Mocna strona #1

Separacja logiki biznesowej od GUI

Logika sterowania i modele matematyczne są całkowicie niezależne od interfejsu graficznego. Dobrym przykładem jest klasa `Petla_Sprzezenia`, która przyjmuje po prostu referencje do czystych klas C++ i wykonuje obliczenia, nie dołączając żadnych bibliotek Qt poza własnymi nagłówkami.

```
#include "Petla_Sprzezenia.h"

Petla_Sprzezenia::Petla_Sprzezenia(ModelARX &arx, RegulatorPID &pid)
    : model(arx)
    , regulator(pid)
    , ostatnie_wyjście(0.0)
{}

double Petla_Sprzezenia::wykonaj_krok(double wartosc_zadana, double dt)
{
    double e = wartosc_zadana - ostatnie_wyjście;
    double u = regulator.symuluj(e, dt);
    double y = model.symuluj(u);
    ostatnie_wyjście = y;
    return y;
}
```



Mocna strona #2

Prawidłowe zarządzanie pamięcią przeszłości przy zmianie parametrów w czasie trwania programu

```
void ModelARX::setA(const std::vector<double> &a)
{
    if (a == wspolczynniki_A)
        return;

    // Przepisanie starej historii do nowego wektora (zapobiega skokom/resetowi)
    std::vector<double> nowaHistoria(a.size(), 0.0);
    size_t doSkopiowania = std::min(historia_Y.size(), nowaHistoria.size());
    for (size_t i = 0; i < doSkopiowania; ++i)
        nowaHistoria[i] = historia_Y[i];

    historia_Y = nowaHistoria;
    wspolczynniki_A = a;
}
```

1 oszczędza procesor

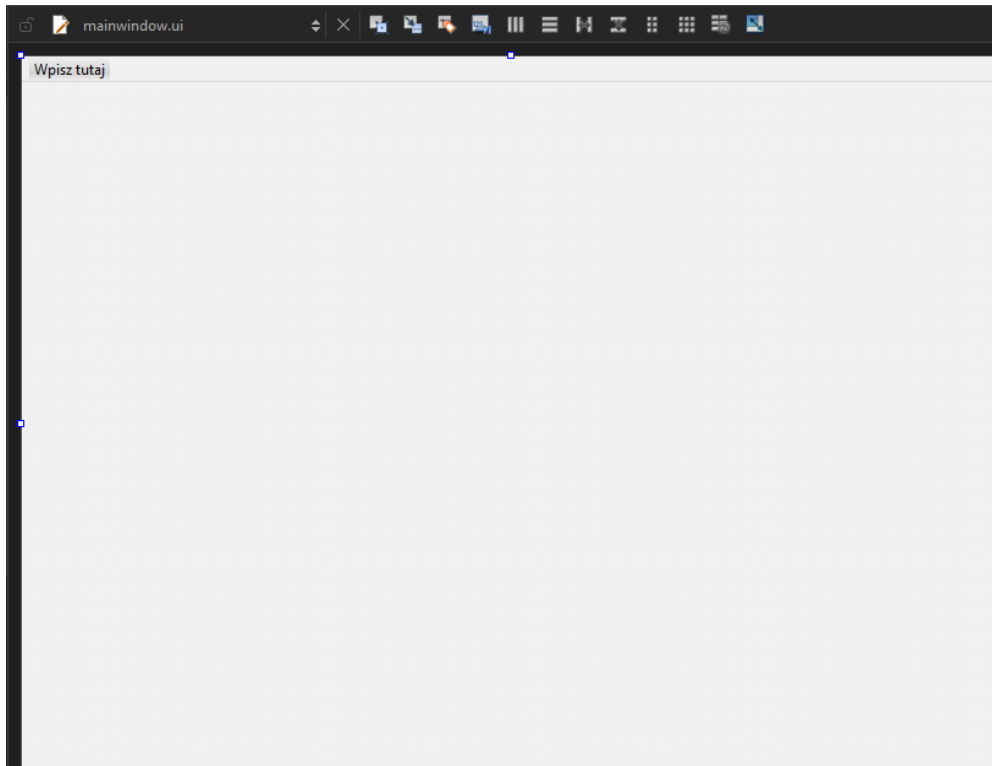
2 chroni program przed wyłączeniem, ponieważ używa `std::min` do bezpiecznego skopiowania pamięci bez ryzyka wyjścia poza zakres tablicy

3 zachowuje wcześniejszą historię symulacji, dzięki czemu po wpisaniu nowych danych wykres płynnie idzie dalej i nie ma na nim sztucznych skoków do zera



Słaba strona #1

Budowa interfejsu graficznego w całości z poziomu kodu C++ w konstruktorze, a niewykorzystanie Qt Designera



```
//LEWY PANEL (KONTROLKI)
QWidget *lewyPanelContainer = new QWidget();
QVBoxLayout *panelSterowania = new QVBoxLayout(lewyPanelContainer);

// Grupa: Sterowanie Symulacją
QGroupBox *grpSym = new QGroupBox("Symulacja");
QFormLayout *layoutSym = new QFormLayout();

spinOknoCzasowe = new QDoubleSpinBox();
spinOknoCzasowe->setRange(5.0, 50.0);
spinOknoCzasowe->setValue(10.0);
spinOknoCzasowe->setSuffix(" s");
spinOknoCzasowe->setSingleStep(1.0);
connect(spinOknoCzasowe,
        &QDoubleSpinBox::editingFinished,
        this,
        &MainWindow::aktualizujOknoCzasowe);

spinInterwal = new QSpinBox();
spinInterwal->setRange(10, 1000);
spinInterwal->setValue(200);
```

```
connect(spinGenOffset, &QDoubleSpinBox::editingFinished, this, &MainWindow::aktualizujGenerator);
connect(spinGenAmp, &QDoubleSpinBox::editingFinished, this, &MainWindow::aktualizujGenerator);
connect(spinGenOkres, &QDoubleSpinBox::editingFinished, this, &MainWindow::aktualizujGenerator);
```



Słaba strona #2

ProstyUAR i Petla_Sprzezenia robią dokładnie to samo łączą regulator z modelem w pętlę zamkniętą.

```
#ifndef PROSTYUAR_H
#define PROSTYUAR_H

#include "ModelARX.h"

class RegulatorPID;

class ProstyUAR
{
public:
    ProstyUAR(ModelARX &arx, RegulatorPID &pid);

    // Główna funkcja symulacji pętli zamkniętej
    // w - wartość zadana
    // Zwraca y - wyjście z obiektu
    double symuluj(double w);

private:
    ModelARX &model;
    RegulatorPID &regulator;
    double poprzednie_y; // Do obliczenia uchybu  $e = w - y(k-1)$ 
};

#endif // PROSTYUAR_H
```

```
#include "Petla_Sprzezenia.h"

Petla_Sprzezenia::Petla_Sprzezenia(ModelARX &arx, RegulatorPID &pid)
    : model(arx)
    , regulator(pid)
    , ostatnie_wyjście(0.0)
{}

double Petla_Sprzezenia::wykonaj_krok(double wartosc_zadana, double dt)
{
    double e = wartosc_zadana - ostatnie_wyjście;
    double u = regulator.symuluj(e, dt);
    double y = model.symuluj(u);
    ostatnie_wyjście = y;
    return y;
}
```

- łamie zasadę DRY
- utrudnia rozwój aplikacji zmuszając programistę do nanoszenia tych samych poprawek w dwóch miejscach naraz
- Petla_Sprzezenia poprawnie uwzględnia krok dt , a ProstyUAR sztywno zakłada 1.0, co prowadzi do całkowicie sprzecznych wyników symulacji.

Projekt protokołu sieciowego

- Zdecydowaliśmy się na protokół TCP z rozdzieleniem symulacji na dwie instancje. Aplikacja regulatora wysyła przez sieć sygnał sterujący, a aplikacja obiektu symuluje sam obiekt regulacji i odsyła z powrotem wartość regulowaną. Transmisję podzieliliśmy na dwa etapy: do przesłania początkowej konfiguracji obiektów użyjemy czytelного formatu tekstowego JSON, natomiast do ciągłego przesyłania próbek symulacji wykorzystamy szybką serializację binarną

Budowa ramki

Ponieważ sieć TCP przesyła dane jako ciągły strumień, każdą naszą wiadomość podzieliliśmy na dwie części:

- **Nagłówek:** to krótka informacja doklejana na samym początku wiadomości (w roli etykiety). Określa ona typ przesyłanych danych (wskazuje, czy dalej znajduje się konfiguracja, sterowanie, czy wartość regulowana) oraz podaje całkowity rozmiar paczki w bajtach. Dzięki temu aplikacja odbierająca dokładnie wie, jak zinterpretować nadchodzącą wiadomość i ile danych musi odczytać z sieci.
- **Ładunek:** to właściwa zawartość paczki, czyli nasze zserializowane dane (tekst JSON z ustawieniami lub binarne próbki do wykresów), które są odczytywane tuż za nagłówkiem.

Nasz protokół dzieli komunikację na dwa typy wiadomości:

1. Paczka konfiguracyjna wysyłana na start i przy zmianie nastaw:

- format tekstowy (JSON): wybrany ze względu na czytelność kodu i elastyczność
- zawiera pełny zrzut stanu obiektów: zmienne modelu ARX (wektory A, B, opóźnienie) oraz nastawy regulatora PID automatycznie przesyłana, gdy po którejkolwiek stronie nastąpi modyfikacja parametrów

2. Ramka symulacyjna wysyłana co krok pętli:

- format binarny: wybrany dla maksymalnej szybkości transmisji i minimalizacji opóźnień.
- zawiera bieżące dane symulacji: sygnał sterujący $u(k)$ (kierunek: regulator $\rightarrow$ obiekt) lub wartość regulowaną $y(k)$ (kierunek: obiekt $\rightarrow$ regulator)
- zawiera identyfikator próbki ID: potrzebny do monitorowania opóźnień sieciowych i wykrywania ewentualnych desynchronizacji symulacji.

System sprawdzania stabilności symulacji i diagnostyka

- Aby upewnić się, że komunikacja wyrabia się w zadanym interwale symulacji, zaimplementujemy system kontroli czasu oparty na ID próbki. Regulator przy wyzwoleniu kolejnego taktu zegara sprawdza, czy otrzymał odpowiedź od obiektu z adekwatnym ID. Jeśli dane nie dotrą na czas, system nie przerywa działania, ale sygnalizuje operatorowi utratę synchronizacji (naprzykład przez komunikat). Dodatkowo zaimplementujemy system mierzący szybkość transmisji, wyświetlający na bieżąco liczbę poprawnie wymienionych pakietów na sekundę, co ułatwia ocenę wydajności zastosowanej serializacji binarnej

Działanie protokołu

- Po nawiązaniu połączenia TCP, instancja regulatora oraz instancja obiektu automatycznie synchronizują swoje ustawienia za pomocą formatu JSON. Każda późniejsza zmiana parametrów, niezależnie po której stronie nastąpi, wyzwała natychmiastową aktualizację u partnera. Po uruchomieniu symulacji, aplikacje przechodzą do ciągłej, dwukierunkowej wymiany binarnych próbek symulacyjnych.

